```
print("Experiment 7")
print("Parth Mahesh Madrewar")
print("Reg. No. : 241080036")
```

Show command palette (⌘/Ctrl+Shift+P)

```
Experiment 7
Parth Mahesh Madrewar
Reg. No. : 241080036
```

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# a. Read the dataset from CSV file
df = pd.read_csv("/content/mushrooms.csv")
print(df.head())
print("Shape:", df.shape)
```

```
  class cap-shape cap-surface cap-color bruises odor gill-attachment  \
0     p         x           s         n       t    p               f
1     e         x           s         y       t    a               f
2     e         b           s         w       t    l               f
3     p         x           y         w       t    p               f
4     e         x           s         g       f    n               f

  gill-spacing gill-size gill-color  ... stalk-surface-below-ring  \
0            c         n          k  ...                        s
1            c         b          k  ...                        s
2            c         b          n  ...                        s
3            c         n          n  ...                        s
4            w         b          k  ...                        s

  stalk-color-above-ring stalk-color-below-ring veil-type veil-color  \
0                      w                      w         p          w
1                      w                      w         p          w
2                      w                      w         p          w
3                      w                      w         p          w
4                      w                      w         p          w

  ring-number ring-type spore-print-color population habitat
0           o         p                 k          s       u
1           o         p                 n          n       g
2           o         p                 n          n       m
3           o         p                 k          s       u
4           o         e                 n          a       g

[5 rows x 23 columns]
Shape: (8124, 23)
```

```
# Preprocessing
# stalk-root has '?' for missing values, keeping it as its own category
X = df.drop("class", axis=1)
y = df["class"]
y = y.map({'e': 0, 'p': 1})     # e -> edible (0), p -> poisonous (1)

X = pd.get_dummies(X, drop_first=True)
X = X.astype(float).values
y = y.astype(float).values

print("Features after encoding:", X.shape[1])
```

```
Features after encoding: 95
```

```
# b. Divide the dataset for learning the task of classification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 6499
Testing samples: 1625
```

```
class NeuralNetworkScratch:
    def __init__(self, input_size, hidden_size, learning_rate=0.5, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.loss = []
        self.test_loss = []
```

```
        # Xavier initialization (suits sigmoid)
        np.random.seed(42)
        self.W1 = np.random.randn(input_size, hidden_size) * np.sqrt(1 / input_size)
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.randn(hidden_size, 1) * np.sqrt(1 / hidden_size)
        self.b2 = np.zeros((1, 1))

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

    def forward(self, X):
        Z1 = np.dot(X, self.W1) + self.b1
        A1 = self.sigmoid(Z1)
        Z2 = np.dot(A1, self.W2) + self.b2
        A2 = self.sigmoid(Z2)
        return A1, A2

    def compute_loss(self, y, y_pred):
        epsilon = 1e-8
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        return -np.mean(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))   #-> binary cross entropy

    def fit(self, X, y, X_val, y_val):
        rows = X.shape[0]
        y = y.reshape(-1, 1)
        y_val = y_val.reshape(-1, 1)

        for i in range(self.epochs):
            # Forward propagation
            A1, A2 = self.forward(X)

            # Error on train and test data
            self.loss.append(self.compute_loss(y, A2))
            self.test_loss.append(self.compute_loss(y_val, self.forward(X_val)[1]))

            # Backpropagation
            dZ2 = A2 - y
            dW2 = np.dot(A1.T, dZ2) / rows
            db2 = np.sum(dZ2, axis=0, keepdims=True) / rows

            dZ1 = np.dot(dZ2, self.W2.T) * (A1 * (1 - A1))
            dW1 = np.dot(X.T, dZ1) / rows
            db1 = np.sum(dZ1, axis=0, keepdims=True) / rows

            # Update weights
            self.W2 = self.W2 - self.learning_rate * dW2
            self.b2 = self.b2 - self.learning_rate * db2
            self.W1 = self.W1 - self.learning_rate * dW1
            self.b1 = self.b1 - self.learning_rate * db1

    def predict(self, X):
        A1, A2 = self.forward(X)
        return (A2 >= 0.5).astype(int).flatten()
```

```
start = time.time()
nn = NeuralNetworkScratch(input_size=X_train.shape[1], hidden_size=16, learning_rate=1.0, epochs=1000)
nn.fit(X_train, y_train, X_test, y_test)
nn_time = time.time() - start

y_pred_nn = nn.predict(X_test)
nn_accuracy = accuracy_score(y_test, y_pred_nn)

print("Scratch Neural Network")
print("Accuracy :", nn_accuracy * 100)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_nn))
print("\nFinal train loss:", nn.loss[-1])
print("Final test loss :", nn.test_loss[-1])
```

```
Scratch Neural Network
Accuracy : 100.0

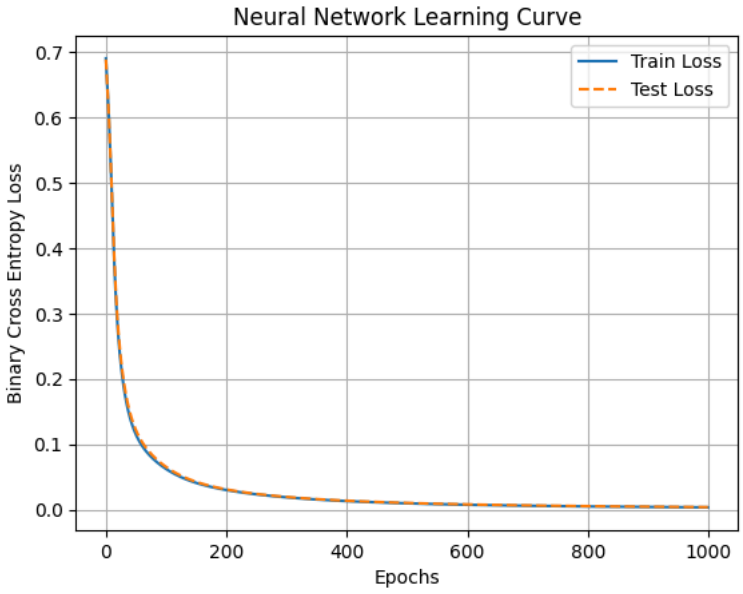
Confusion Matrix:
[[843   0]
 [  0 782]]

Final train loss: 0.0037013161197325434
Final test loss : 0.00406705097809383
```

```
# c. Plot the graph of error
plt.plot(nn.loss, label="Train Loss")
```

```
plt.plot(nn.test_loss, label="Test Loss", linestyle="--")
plt.title("Neural Network Learning Curve")
plt.xlabel("Epochs")
plt.ylabel("Binary Cross Entropy Loss")
plt.legend()
plt.grid()
```

Show command palette (⌘/Ctrl+Shift+P)




```
# d. Decision Tree using Scikit-Learn
start = time.time()
tree_model = DecisionTreeClassifier(criterion="entropy", random_state=42)
tree_model.fit(X_train, y_train)
tree_time = time.time() - start

y_pred_tree = tree_model.predict(X_test)
tree_accuracy = accuracy_score(y_test, y_pred_tree)

print("Scikit-Learn Decision Tree")
print("Accuracy :", tree_accuracy * 100)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_tree))
```

```
Scikit-Learn Decision Tree
Accuracy : 100.0

Confusion Matrix:
[[843   0]
 [  0 782]]
```

```
# Comparison
cm_nn = confusion_matrix(y_test, y_pred_nn)
cm_tree = confusion_matrix(y_test, y_pred_tree)

print("Metric                    Neural Network    Decision Tree")
print("Accuracy (%)              ", round(nn_accuracy * 100, 2), "          ", round(tree_accuracy * 100, 2))
print("Poisonous predicted edible", "     ", cm_nn[1][0], "                ", cm_tree[1][0])
print("Training time (s)         ", round(nn_time, 3), "         ", round(tree_time, 3))

print("\nTree depth :", tree_model.get_depth())
print("Tree leaves:", tree_model.get_n_leaves())
print("NN parameters:", nn.W1.size + nn.b1.size + nn.W2.size + nn.b2.size)
```

```
Metric                    Neural Network    Decision Tree
Accuracy (%)               100.0            100.0
Poisonous predicted edible       0                  0
Training time (s)          15.059           0.02

Tree depth : 5
Tree leaves: 11
NN parameters: 1553
```

Comparison and Comments

Result: Both the scratch neural network and the Scikit-Learn decision tree reach (almost) perfect accuracy on the test set, with no poisonous mushroom predicted as edible.

Why both are so accurate: The mushroom dataset is almost perfectly separable. A few features (especially `odor`) already separate edible and poisonous mushrooms, so a shallow tree (depth 5, 11 leaves) is enough. A 100% score therefore says more about the dataset than about the algorithm.

Learning curve: The training loss falls smoothly with epochs, and the test loss follows it closely. There is no gap between the two curves, so the network is not overfitting.

Show command palette (⌘/Ctrl+Shift+P)

Training cost: The decision tree trains in milliseconds. The neural network needs 1000 epochs of full-batch gradient descent and is far slower. It also depends on the learning rate, the number of hidden neurons and the weight initialization. With a bad learning rate the loss oscillates or fails to converge.

Interpretability: The tree gives readable if-else rules, so we can see which feature decides the class. The network has about 1500 weights and works as a black box.

Conclusion: For this small, categorical, almost separable dataset the decision tree is the better choice, because it is faster, simpler and interpretable, with the same accuracy. The neural network would be preferable for larger or less separable data where relationships between features are non-linear and complex.